

Rodrigo Briz Ramos

Isaac Pérez Mascaró

3º Informática industrial y Robótica. Visión por Computador

Curso 25/26

PROYECTO FINAL DE LA ASIGNATURA- Reconocimiento de Señales GTSRB

1. Introducción y Objetivos

En este proyecto hemos desarrollado un sistema automático para la detección y clasificación de señales de tráfico, centrándonos en cinco categorías específicas del conjunto de datos GTSRB (German Traffic Sign Recognition Benchmark): Ceda el Paso (13), Stop (14), Prohibido el paso (17), Giro obligatorio a la derecha (33) y Mantenimiento del sentido a la derecha (38). Estas señales presentan una combinación de formas geométricas (triangulares, circulares y octogonales) y colores (rojo y azul) que suponen un reto equilibrado para el análisis de imagen.

El objetivo principal que nos hemos propuesto es implementar un "pipeline" completo de procesamiento utilizando exclusivamente técnicas de Visión por Computador. No hemos utilizado redes neuronales para así demostrar la eficacia de los métodos de procesamiento digital de imágenes, segmentación por color, operaciones morfológicas y extracción de características mediante descriptores de gradientes orientados (HOG) combinados con aprendizaje supervisado tradicional.

Con este enfoque, buscamos no solo obtener una alta tasa de acierto en la clasificación, sino también construir un sistema robusto que sea capaz de preprocesar y filtrar la información visual de entrada para trabajar únicamente con regiones de interés que contengan información útil para el clasificador.



2. Descripción del Pipeline Propuesto

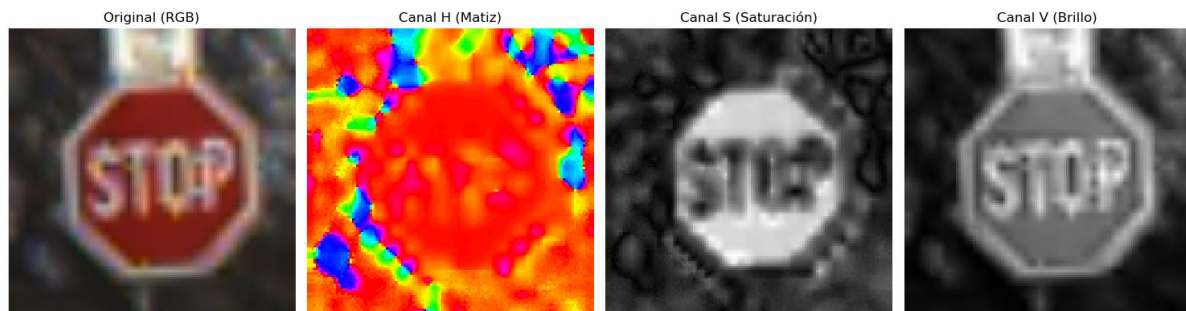
Para abordar el problema de clasificación, hemos diseñado un flujo de trabajo secuencial que transforma la imagen en bruto en un vector de características altamente representativo. Este

proceso se divide en tres fases fundamentales: preprocesamiento, segmentación morfológica y extracción de descriptores.

2.1. Preprocesamiento y Normalización

En la primera etapa, cargamos las imágenes y las redimensionamos a un tamaño estándar de 64×64 píxeles para asegurar la uniformidad en los descriptores posteriores. Seguidamente, aplicamos un suavizado Gaussiano con un kernel de 5×5 para reducir el ruido de alta frecuencia que podría generar falsos positivos durante la detección de bordes o la segmentación.

Finalmente, realizamos una conversión del espacio de color BGR al espacio HSV (Hue-Saturation-Value). Hemos seleccionado este espacio de color por su capacidad para separar la información cromática (matiz) de la luminancia, lo que nos permite definir rangos de color mucho más robustos frente a sombras o variaciones en la exposición lumínica de las escenas.



2.2. Segmentación Cromática y Limpieza Morfológica

Una vez en el espacio HSV, aislamos los píxeles pertenecientes a la señal mediante umbralización por color. Para las señales rojas (13, 14, 17), implementamos un rango doble para capturar los tonos rojos en ambos extremos del espectro angular del matiz. Para las azules (33, 38), utilizamos un rango centrado en los tonos azulados.

Color	Rango H (Matiz)	Rango S (Saturación)	Rango V (Valor)
Rojo	0 - 10	70 - 255	50 - 255
Rojo	170 - 180	70 - 255	50 - 255
Azul	100 - 140	150 - 255	0 - 255

Tras obtener la máscara binaria inicial, aplicamos **operaciones morfológicas** esenciales para refinar la región de interés:

- **Apertura:** Mediante un elemento estructurante de 5×5, eliminamos pequeñas motas de ruido aisladas en el fondo de la imagen.

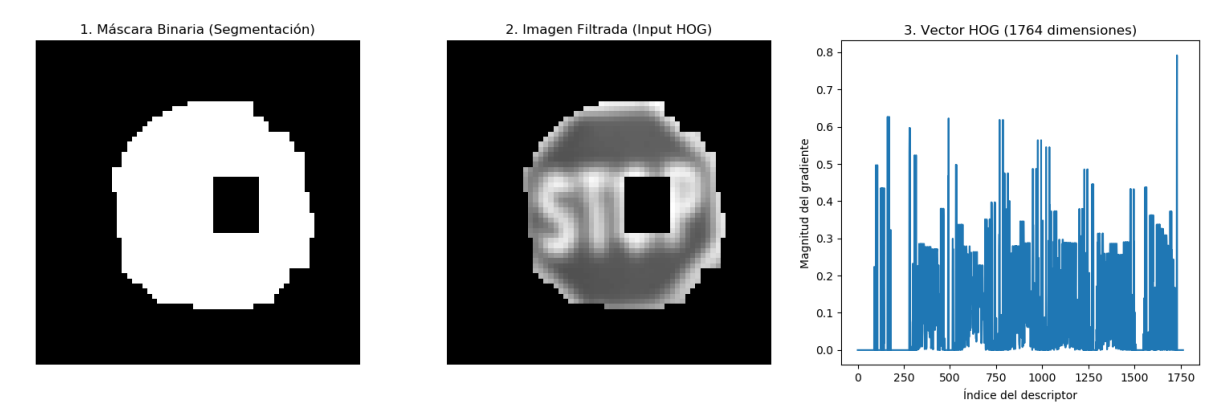
- **Cierre:** Con el mismo kernel, conectamos componentes y rellenamos posibles huecos internos en la máscara, obteniendo una silueta sólida y fiel a la forma real de la señal.

En este punto del pipeline, hemos introducido un criterio de validación: si tras la limpieza la máscara resultante no contiene píxeles activos, la muestra se descarta automáticamente. Esto garantiza que el clasificador solo reciba información de imágenes donde la señal ha sido correctamente localizada.

2.3. Extracción de Características (HOG)

Con la máscara limpia, aplicamos una operación AND bit a bit sobre la versión en escala de grises de la imagen suavizada para aislar el contenido de la señal. Sobre esta región extraemos el descriptor HOG (Histogram of Oriented Gradients).

Hemos configurado el descriptor con una ventana de 64×64 , bloques de 16×16 y celdas de 8×8 , calculando 9 orientaciones de gradiente. Esta técnica es idónea para nuestro caso, ya que codifica la estructura y la geometría interna de las señales (como las flechas o los bordes de la señal de Stop) en un vector unidimensional de 1764 valores que servirá de entrada para el modelo de aprendizaje.



2.4. Clasificación y Reconocimiento

En esta fase final del desarrollo, hemos implementado la lógica de aprendizaje necesaria para que el sistema sea capaz de distinguir las categorías de señales de forma autónoma.

2.4.1. Entrenamiento del Modelo (SVM)

Tras la extracción de los vectores HOG de las 417 muestras que superaron el filtrado inicial, procedimos a la creación del clasificador. Hemos seleccionado una Máquina de Vectores de Soporte (SVM) configurada con un kernel lineal y un parámetro de regularización $C=1.0$. Esta elección se fundamenta en que los descriptores HOG suelen ser linealmente separables en espacios de alta dimensionalidad (1764 valores por vector), y el kernel lineal ofrece un equilibrio óptimo entre la capacidad de generalización del modelo y su eficiencia computacional.

2.4.2. Evaluación y Validación

Para validar correctamente los resultados, hemos dividido el conjunto de datos filtrado en dos bloques independientes mediante un muestreo estratificado. Hemos destinado un 80% de las muestras para el entrenamiento del modelo y un 20% (correspondiente a 84 imágenes) para el conjunto de test. Esta separación estricta asegura que las métricas de rendimiento obtenidas reflejen fielmente el comportamiento del sistema ante imágenes que nunca han sido procesadas por el algoritmo durante la fase de aprendizaje.

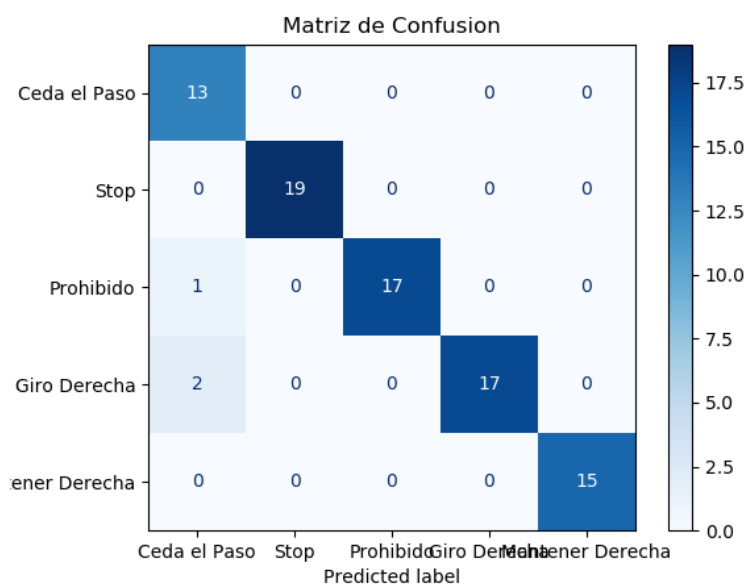
3. Resultados Experimentales y Visuales

3.1. Análisis Estadístico y Métricas

Partiendo de 500 imágenes iniciales, el sistema identificó 417 muestras útiles tras el preprocesamiento. El descarte de 83 imágenes se debe a condiciones lumínicas extremas que impedían una segmentación fiable en HSV, actuando como un filtro de calidad para el clasificador. En el conjunto de test, hemos alcanzado una precisión global (Accuracy) del 96.43%, clasificando correctamente 81 de las 84 señales.

3.2. Matriz de Confusión

La matriz de confusión muestra que la diagonal principal concentra casi todas las predicciones, alcanzando una **precisión global del 96,43%**. Se observa una excelente discriminación en las señales de **'Stop' (19/19)**, **'Mantener Derecha' (15/15)** y **'Ceda el Paso' (13/13)**, categorías donde la estructura geométrica es muy clara y los gradientes HOG son muy distintivos. No obstante, se detecta una ligera confusión en las señales circulares (Prohibido y Giro Derecha), que en 3 ocasiones fueron clasificadas erróneamente como Ceda el Paso, probablemente debido a un suavizado excesivo de los bordes en imágenes de baja resolución.



3.3. Visualización de Aciertos



4. Discusión y Conclusiones

4.1. Análisis de Fallos

Hemos detectado 3 errores en el test. Al analizar la visualización de fallos, observamos que señales circulares ("Prohibido" y "Giro Derecha") fueron confundidas con la señal triangular de "Ceda el Paso". Esto ocurre en imágenes con una resolución extremadamente baja o desenfoque de movimiento, donde los gradientes HOG se suavizan y pierden la nitidez necesaria para distinguir un círculo de un triángulo invertido.



4.2. Conclusión

Hemos logrado desarrollar un sistema funcional que cumple con los requisitos del proyecto utilizando exclusivamente visión clásica. La robustez del espacio **HSV** para la segmentación y la potencia estructural del **HOG** han permitido superar el 96% de acierto, demostrando que estos métodos siguen siendo eficaces para la señalización vial.

5. Manual Técnico de Despliegue y Ejecución

Para garantizar la reproducibilidad de los resultados experimentales presentados en este informe, se detallan a continuación los requisitos de software, la estructura de directorios necesaria y los pasos para la ejecución del sistema.

5.1. Requisitos del Entorno

El proyecto ha sido desarrollado e implementado bajo el lenguaje **Python (versión 3.8 o superior)**. Para su correcto funcionamiento, es necesario disponer de las siguientes bibliotecas de terceros instaladas en el entorno:

- **OpenCV (opencv-python):** Utilizada para la lectura de imágenes, conversión de espacios de color (HSV), operaciones morfológicas y visualización de resultados.

- **NumPy (numpy):** Fundamental para el manejo eficiente de matrices de píxeles y operaciones vectoriales.
- **Scikit-learn (scikit-learn):** Proporciona la implementación del clasificador SVM (Support Vector Machine), las herramientas de división del dataset y el cálculo de métricas de evaluación.
- **Matplotlib (matplotlib):** Empleada para la generación y guardado de la matriz de confusión y gráficos de descriptores.

Comando de instalación rápida:

Bash

```
pip install opencv-python numpy scikit-learn matplotlib
```

5.2. Estructura del Proyecto

El sistema utiliza rutas relativas para localizar las imágenes, por lo que es crítico mantener la siguiente jerarquía de directorios. El dataset debe encontrarse en un nivel superior a la carpeta del código fuente para simular un entorno de desarrollo real.

Plaintext

Proyecto_Vision_RodrigoBriz/

```
|
|— dataset/      # Carpeta con las imágenes originales
| |— 13/         # Ceda el Paso (.ppm)
| |— 14/         # Stop (.ppm)
| |— 17/         # Prohibido el Paso (.ppm)
| |— 33/         # Giro Derecha (.ppm)
| |— 38/         # Mantener Derecha (.ppm)
|
|— codigo/       # Carpeta con el código fuente
| |— main.py     # Script principal de ejecución
| |— preprocesamiento.py # Funciones de carga, resize y HSV
| |— segmentacion.py  # Máscaras de color y morfología
| |— descriptores.py  # Extracción de características HOG
| |— clasificador.py  # Entrenamiento del SVM
| |— evaluacion.py    # Cálculo de métricas y visualización
```

5.3. Instrucciones de Ejecución

Para iniciar el proceso de entrenamiento y evaluación, siga estos pasos:

Abra una terminal de comandos.

Navegue hasta el directorio que contiene los scripts:

```
cd Proyecto_Visión_RodrigoBriz/codigo
```

Ejecute el script principal:

```
python3 main.py
```

5.4. Interacción y Resultados Esperados

Una vez lanzado el programa, el sistema realizará las siguientes acciones de forma automática:

1. **Procesamiento en Terminal:** Se mostrará una barra de progreso indicando la carga de imágenes, el filtrado de muestras no válidas y el entrenamiento del modelo. Finalmente, imprimirá la **Precisión Global (Accuracy)** obtenida.
2. **Generación de Archivos:** Se creará automáticamente un archivo llamado `confusion_matrix.png` en la carpeta de ejecución, que contiene el gráfico de rendimiento por clases.
3. **Visualización:** Se abrirán dos ventanas emergentes de OpenCV:
 - a. **"Ejemplos Correctos":** Muestra un mosaico de señales clasificadas exitosamente (recuadro verde).
 - b. **"Ejemplos Fallidos":** Muestra las señales donde el modelo erró (recuadro rojo), útil para el análisis de errores.
4. **Finalización:** Para cerrar las ventanas y terminar la ejecución del programa, pulse **cualquier tecla** sobre las ventanas de imagen.

5.5 Video demostrativo

https://drive.google.com/file/d/1aMTJ9HDZD54SIh_zE8nBq_r7W6t1mpMM/view?usp=sharing